ANALYSIS REPORT

Student Record Management System

Data Structures & Algorithms | C++ Programming Project

Project Title	Student Record Management System
Language	C++ (Standard: C++17)
Data Structure	std::vector (Dynamic Array)
Operations	Add · Display · Search · Delete · Sort · Statistics
Analysis Types	RAM Model · Time Complexity · Space Complexity
Submitted By	Muhammad Yousaf
Student ID	66160
Course	Analysis of Algorithm
Instructor	Maueed Ahmed Mirza
Date	5/9/2026

Table of Contents

Introduction & Project Overview 3
1.1 Objectives 3
1.2 Feature Summary 3
2 | System Architecture 3
2.1 Data Model 3
2.3 Sorting Algorithm — Introsort 4
3 | RAM Model Analysis 4
3.1 RAM Model Assumptions 4
3.2 Operation-by-Operation RAM Counts 5
3.3 Key RAM Model Observations 5
4 | Time Complexity Analysis 5
4.1 addStudent — O(n) 5
T(n) = O(n) [dup check] + O(1) [push_back] = O(n) 5
4.2 displayAll — O(n) 6
4.3 searchByID — O(n) worst case 6
Best: O(1) | Average: O(n/2) = O(n) | Worst: O(n) 6
4.4 searchByName — O(n) worst case 6
4.5 deleteStudent — O(n) 6
T(n) = O(n) [search] + O(n) [erase/shift] = O(n) 6
4.6 sortByGPA / sortByName / sortByID — O(n log n) 6
T(n) = O(n log n) — best, average, and worst case 6
4.7 displayStats — O(n) 7
5 | Space Complexity Analysis 7
5.1 Data Storage — O(n) 7
5.2 Auxiliary Space per Operation 7
5.3 Overall Space Complexity 7
6 | Complete Complexity Summary Table 8
6.1 Complexity Distribution 8
7 | Conclusion 8
7.1 Key Takeaways 9

Introduction & Project Overview

This report presents a comprehensive algorithmic analysis of a **Student Record Management System** implemented in C++17. The system is built around the **std::vector** (dynamic array) data structure and supports full CRUD operations alongside sorting and statistical queries. The analysis covers three formal frameworks: the **RAM Model**, **Time Complexity**, and **Space Complexity**.

1.1 Objectives

- Implement a functional student record system using C++17.
- Select an appropriate data structure and justify the choice analytically.
- Apply the RAM (Random Access Machine) model to count primitive operations.
- Derive asymptotic time complexity for every operation using Big-O notation.
- Measure auxiliary and total space complexity of the system.

1.2 Feature Summary

Feature	Description	Menu Option
Add Student	Insert a new student record with duplicate-ID check	[1]
Display All	Print all records in a formatted table	[2]
Search by ID	Linear scan to find a specific student by integer ID	[3]
Search by Name	Case-insensitive substring search	[4]
Delete Student	Remove a record by ID; shift remaining elements	[5]
Sort by GPA	Sort descending by GPA using std::sort (introsort)	[6]
Sort by Name	Sort ascending alphabetically using std::sort	[7]
Sort by ID	Sort ascending by student ID using std::sort	[8]
Statistics	Compute min, max, and average GPA in a single pass	[9]

Table 1 – System Feature Summary

2 | System Architecture

2.1 Data Model

Each student is represented as a plain **struct** containing three fields. Under the RAM model, reading or writing any field costs exactly one operation.

```
struct Student {     int    id;       // 4 bytes  — unique integer
identifier     string name;     // heap-allocated, bounded length
 → O(1) per field     double gpa;      // 8 bytes  — grade point
                      average [0.0 – 4.0] };
```

2.2 Why std::vector?

A **std::vector** was chosen as the primary container for the following reasons:

- **O(1) random access** — any element can be read or written in constant time via its index, which satisfies the RAM model's unit-cost memory assumption.
- **O(1) amortised insertion** — push_back doubles capacity geometrically, keeping the amortised per-insertion cost at O(1).
- **Cache locality** — elements are stored contiguously in memory, improving real-world performance compared to linked structures.
- **Simplicity** — the vector's flat layout makes complexity analysis straightforward and pedagogically clear.

2.3 Sorting Algorithm — Introsort

All three sort operations delegate to **std::sort**, which implements **introsort** — a hybrid algorithm combining:

- **Quicksort** for average-case speed (O(n log n) average).
- **Heapsort** fallback when recursion depth exceeds 2·log n, guaranteeing O(n log n) worst-case. 2
- **Insertion sort** for small partitions (≤ 16 elements), which is faster in practice due to low overhead.

Note: Unlike plain quicksort, introsort has no $O(n^2)$ worst case, making it suitable for all input distributions.

3 | RAM Model Analysis

The **Random Access Machine (RAM)** model is a formal computational model used to count the exact number of primitive operations executed by an algorithm, independent of hardware.

3.1 RAM Model Assumptions

- Each **basic operation** (read, write, add, subtract, multiply, divide, compare) costs exactly **1 unit** of time.
- **Memory access** at any valid index is O(1) — there is no cache hierarchy or memory latency.
- Instructions execute **sequentially**; no parallelism.
- A loop of n iterations with a constant-cost body costs exactly **n units**.

3.2 Operation-by-Operation RAM Counts

Operation	RAM Count	Breakdown of Operations
`addStudent`	`≤ 2n + 4`	n comparisons (dup check) + 3 field writes + 1 push_back
`displayAll`	`n`	1 read per element × n elements
`searchByID`	`≤ n`	≤ n comparisons; stops at first match
`searchByName`	`≤ n`	≤ n string comparisons (bounded length)
`deleteStudent`	`≤ 2n − 1`	≤ n comparisons (find) + ≤ n−1 writes (erase/shift)
`sortByGPA/Name/ID`	`c·n·log n`	c·n·log n comparisons (introsort, small constant c)
`displayStats`	`3n + 1`	3 comparisons/additions per element + 1 division

Table 2 – RAM Operation Counts

3.3 Key RAM Model Observations

- All single-step operations (read, write, compare) are assigned cost 1.
- A loop with n iterations and a constant-cost body = n total RAM operations.
- For **deleteStudent**, the find phase costs up to n operations and the erase/shift phase costs up to n−1, giving ≤ 2n − 1 total.
- For **displayStats**, each element requires exactly 3 unit operations (min compare, max compare, sum add), plus 1 final division → 3n + 1 total.
- Sorting performs c·n·log n comparisons where c is a small implementation constant that disappears inBig-O notation.
-

4 | Time Complexity Analysis

Time complexity measures how the running time of an algorithm scales with the size of the input n. We use **Big-O notation** to express asymptotic upper bounds. Here n represents the number of student records stored in the system.

4.1 addStudent — O(n)

Adding a student involves two phases: a **duplicate ID scan** over all n existing records (O(n)), followed by a vector push_back (O(1) amortised).

```
T(n) = O(n) [dup check] + O(1) [push_back] = O(n)
```

- Best case O(n): still must scan all n records to confirm no duplicate.
- Amortised push_back: the vector doubles capacity geometrically, so the total cost of n insertions is O(n),giving O(1) per insertion amortised.

•

4.2 displayAll — O(n)

Iterates over every element once, calling printStudent (O(1)) per record.

```
T(n) = n × O(1) = O(n)
```

4.3 searchByID — O(n) worst case

Performs a sequential linear scan, comparing each record's ID to the target.

```
Best: O(1) | Average: O(n/2) = O(n) | Worst: O(n)
```

- Best case O(1): target is the first element; loop exits immediately.
- Worst case O(n): target is the last element or absent; all n comparisons made.

4.4 searchByName — O(n) worst case

Case-insensitive substring search; each string comparison is O(1) because name length is bounded by a constant.

```
T(n) = n × O(1) = O(n)
```

- Cannot break early (all matches must be found), so best ≈ worst = O(n).

4.5 deleteStudent — O(n)

Two-phase operation: linear scan to locate the target (O(n)) plus vector::erase which shifts all subsequent elements left (O(n)).

```
T(n) = O(n) [search] + O(n) [erase/shift] = O(n)
```

- vector::erase invalidates all iterators past the deleted position and physically moves up to n−1 elements.
- Deletion from the last position is O(n) for search but O(1) for erase.

4.6 sortByGPA / sortByName / sortByID — O(n log n)

All three sort operations use **std::sort** (introsort). Introsort guarantees O(n log n) in all cases by switching to heapsort when quicksort's recursion depth exceeds 2·log n.

2

```
T(n) = O(n log n) — best, average, and worst case
```

- Quicksort phase: O(n log n) average, O(n²) worst — but introsort never reaches the worst case due to theheapsort fallback.
- Heapsort fallback: O(n log n) worst case guaranteed.
- Insertion sort for small partitions (≤ 16 elements): O(1) per call, improves real-world constants.
- The sort is **in-place**: no extra array is allocated.

•

4.7 displayStats — O(n)

Single-pass computation of minimum, maximum, and average GPA.

```
T(n) = 3n + 1 = O(n)
```

5 | Space Complexity Analysis

Space complexity measures memory usage relative to input size n. We distinguish between **total space** (all memory used by the program) and **auxiliary space** (extra memory beyond the input storage).

5.1 Data Storage — O(n)

The primary data store is a **std::vector<Student>** allocated on the heap. Each Student object occupies a fixed amount of memory:

Field	Type	Size	Notes
`id`	`int`	4 bytes	Fixed-size integer
`name`	`string`	~24 bytes + heap	SSO for short names; bounded max length
`gpa`	`double`	8 bytes	IEEE 754 double-precision float
Total per Student			**Constant — independent of n**

Table 3 – Memory Layout per Student Object

```
Total vector space = n × O(1) = O(n)
```

5.2 Auxiliary Space per Operation

- **addStudent**: O(1) — a few stack variables (id, name, gpa, loop iterator).
- **displayAll**: O(1) — only the loop variable; no extra array.
- **searchByID / searchByName**: O(1) — one boolean flag, one temp string.
- **deleteStudent**: O(1) — one iterator variable.
- **sortByGPA / sortByName / sortByID**: O(log n) — introsort's recursion call stack depth is bounded by 2·log n. No auxiliary array is allocated.
  2
- **displayStats**: O(1) — three scalar accumulators (minGPA, maxGPA, sum).

5.3 Overall Space Complexity

The dominant memory consumer is the student vector itself. All other variables are O(1) scalars on the stack or O(log n) call-stack frames during sorting.

```
Total Space = O(n) [vector] + O(log n) [sort stack] + O(1) [stack vars] =
O(n)
```

Note: O(log n) is asymptotically dominated by O(n) for all n > 1, so the overall space complexity simplifies to O(n).

6 | Complete Complexity Summary Table

The table below consolidates all complexity results for quick reference. n denotes the number of student records stored at the time of the operation.

Operation	Best Case	Average Case	Worst Case	Aux Space
`addStudent`	`O(n)`	`O(n)`	`O(n)`	`O(1)`
`displayAll`	`O(n)`	`O(n)`	`O(n)`	`O(1)`
`searchByID`	`O(1)`	`O(n)`	`O(n)`	`O(1)`
`searchByName`	`O(n)`	`O(n)`	`O(n)`	`O(1)`
`deleteStudent`	`O(n)`	`O(n)`	`O(n)`	`O(1)`
`sortByGPA`	`O(n log n)`	`O(n log n)`	`O(n log n)`	`O(log n)`
`sortByName`	`O(n log n)`	`O(n log n)`	`O(n log n)`	`O(log n)`
`sortByID`	`O(n log n)`	`O(n log n)`	`O(n log n)`	`O(log n)`
`displayStats`	`O(n)`	`O(n)`	`O(n)`	`O(1)`

Table 4 – Full Complexity Summary (n = number of student records)

6.1 Complexity Distribution

Of the 9 operations implemented:

- **6 operations** run in **O(n)** time — linear scans or single-pass iterations.
- **3 operations** run in **O(n log n)** time — all are sort operations using introsort.
- **0 operations** require $O(n^2)$ or worse — the system has no nested loops over n.
- **All operations** use **O(1)** auxiliary space except sorting, which uses O(log n).
- **Overall system space** is **O(n)** — linear in the number of students stored.

7 | Conclusion

This project implemented a fully functional **Student Record Management System** in C++17 using **std::vector** as the underlying data structure. The system supports nine distinct operations covering creation, retrieval, modification, deletion, sorting, and statistical analysis of student records.

The **RAM model analysis** established exact primitive-operation counts for each function, providing a hardware-independent measure of algorithmic cost. The **time complexity analysis** revealed that all non-sort operations are O(n), reflecting the linear nature of unsorted sequential search and traversal. The three sort operations achieve the optimal comparison-sort bound of **O(n log n)** through the use of introsort in std::sort.

The **space complexity analysis** confirmed that the system uses O(n) total memory, dominated by the n student objects stored on the heap. All operations except sorting use O(1) auxiliary space; sorting requires O(log n) call-stack space for the introsort recursion, which is negligible in practice.

7.1 Key Takeaways

- std::vector provides O(1) access and O(1) amortised insertion — ideal for append-heavy workloads whereinsertion order does not matter.
- Linear search (O(n)) is acceptable for small datasets but would benefit from binary search (O(log n)) ifrecords were kept sorted at all times.
- If the system were extended to millions of records, a **hash table** (O(1) average search/delete) or **balanced BST** (O(log n) all operations) would be more appropriate data structures.
- The use of **std::sort** (introsort) ensures worst-case O(n log n) sorting, avoiding the O(n^2) pitfall of naïve quicksort implementations.

End of Analysis Report — Student Record Management System